

DISTRIBUTED SYSTEMS

Course Code: BCE-C621

Batch: 2023–2024 onwards	MM: 100 ESE: 70 Sessional: 30
Credits: 4 L-T-P: 3-1-0	Time: 3 Hrs Total Hours: 40

Prerequisites: Basic computer organization, sequential algorithms, and elementary OS concepts.

Course Objectives

- Understand the foundations of distributed systems.
- Learn issues related to Clock Synchronization and global state in distributed systems.
- Learn distributed mutual exclusion and deadlock detection algorithms.
- Understand the significance of agreement, fault tolerance and recovery protocols.
- Learn characteristics of peer-to-peer and distributed shared memory systems.

1.1 What is a Distributed System?

A **Distributed System** is a collection of independent computers that appears to its users as a single coherent system. The computers communicate only by passing messages over a network. Key characteristics include:

- Concurrency of components
- Lack of a global clock
- Independent failures of components
- Resource sharing over the network

Key Point: No global clock and independent failures are the two fundamental challenges in distributed systems.

1.2 Goals of Distributed Systems

Resource Sharing	Allow users to share hardware, software, and data resources across the network.
Openness	Systems follow standard protocols so components from different vendors can interoperate.
Scalability	System performance should scale up as more users/nodes are added.
Fault Tolerance	System continues to operate correctly even when some components fail.

1.3 Types of Transparency

Transparency hides the complexity of distribution from users and programmers:

Access Transparency	Hide differences in data representation and how resources are accessed.
Location Transparency	Users need not know the physical location of a resource.
Migration Transparency	Resources can be moved without affecting users or applications.
Replication Transparency	Multiple copies of resources exist without user awareness.
Concurrency Transparency	Multiple users share resources without knowing about each other.
Failure Transparency	Failures are concealed; system continues to function.

Exam Tip: All 6 transparency types are commonly asked in short-answer (6-mark) questions. Memorize each with a one-line definition.

1.4 System Models

- **Architectural Model:** Defines software components, their roles, and interconnections. Examples: client-server, peer-to-peer, three-tier architecture.
- **Fundamental Model:** Captures properties relevant to correctness — Interaction (communication), Failure (types of faults), Security (threats and mechanisms).

1.5 Inter-Process Communication (IPC)

Processes communicate via message passing since they do not share memory across machines.

- **Sockets:** Low-level API providing endpoint for communication. TCP sockets (reliable, stream) vs UDP sockets (unreliable, datagram).
- **Message Passing:** send() and receive() primitives — synchronous (blocking) or asynchronous (non-blocking).
- **Shared Memory (local):** Only within same machine; not applicable across network.

1.6 Marshalling & Unmarshalling

Marshalling is the process of converting data structures or objects into a format suitable for transmission over the network. **Unmarshalling** (or deserialization) reconstructs the original data at the receiver. Issues include byte-ordering (big-endian vs little-endian) and data type differences between systems.

1.7 Client–Server Communication

- **Request–Reply Protocol:** Client sends request, server processes and sends reply. Simple and widely used.
- **Group Communication:** One-to-many (multicast) messaging. Used in distributed coordination.
- **Case Study — UNIX IPC:** Pipes, FIFOs, message queues, semaphores, shared memory segments.

Key Point: TCP provides reliable, ordered byte-stream delivery. UDP is faster but unreliable — used where speed matters (DNS, video streaming).

2.1 Remote Procedure Call (RPC)

RPC allows a program to execute a procedure on a remote machine as if it were local. This gives *location transparency* — the caller does not know where the callee runs.

RPC Steps:

- Client calls local stub procedure.
- Stub marshals parameters into a message.
- Message is sent over the network to the server.
- Server stub receives, unmarshals parameters, calls the actual function.
- Result is marshalled and returned to client stub.
- Client stub unmarshals result and returns to caller.

Client Stub	Proxy on client side that packages arguments and sends request.
Server Stub	Proxy on server side that receives request, unpacks arguments, calls function.
Marshalling	Converting parameters into a transmittable byte format.
Binding	Process by which client locates the server (static or dynamic binding).

2.2 Remote Method Invocation (RMI) — Java

Java RMI extends RPC to distributed objects. A remote object can be invoked from a different JVM (on a different machine) as if it were a local object.

- **Remote Interface:** Declares methods accessible remotely (extends `java.rmi.Remote`).
- **RMI Registry:** Naming service — server registers remote object, client looks it up.
- **Stubs & Skeletons:** Auto-generated proxies for communication.
- **RMI vs RPC:** RMI supports object-oriented semantics; RPC is procedure-oriented.

Exam Tip: Know the 4 steps to set up Java RMI: (1) Define remote interface, (2) Implement it, (3) Register with `rmiregistry`, (4) Client lookup and invoke.

2.3 Events & Notifications

In the **publish-subscribe** (event-driven) model, producers publish events and consumers subscribe to event types. This decouples sender and receiver.

- **Event:** occurrence of something noteworthy (e.g., sensor reading, user action).
- **Notification:** message sent to all subscribers when an event occurs.
- **Use cases:** GUI frameworks, stock tickers, IoT, message brokers (Kafka, RabbitMQ).

2.4 OS Support for Distributed Systems

- **Processes & Threads:** OS schedules processes; threads within a process share memory. Multi-threaded servers handle concurrent clients efficiently.
- **Sockets:** OS provides socket API (socket, bind, listen, accept, connect, send, recv).
- **OS Layer Protection:** Kernel mode vs user mode; system calls as the interface.
- **Containers (Docker):** Lightweight OS-level virtualization — isolated process namespaces.

2.5 Quorum-Based Protocol

In a replicated system with N replicas, a **quorum** is the minimum number of replicas that must acknowledge a read/write for it to succeed.

- **Write quorum (W):** number of replicas that must confirm a write.
- **Read quorum (R):** number of replicas from which a read must succeed.
- **Condition for consistency:** $R + W > N$ (ensures overlap between read and write sets).

Key Point: Quorum ensures at least one up-to-date replica is always included in reads. If $N=5$, $W=3$, $R=3$
→ $R+W=6 > 5$ ✓

3.1 Distributed File System (DFS)

A DFS allows users to access files stored on remote servers as if they were local files. Key design goals:

Transparency	Users unaware of file location, replication, or migration.
Concurrency	Multiple clients can access/update files simultaneously with consistency.
Replication	Copies of files on multiple servers for fault tolerance and performance.
Heterogeneity	Works across different OS, hardware, and network types.
Fault Tolerance	Service continues even if some servers fail.
Consistency	All clients see the same view of the file system.
Security	Authentication and access control for files.
Efficiency	Performance comparable to local file access.

3.2 File Service Architecture

A DFS is structured as three components:

- **Flat File Service:** Handles all operations on file contents using unique file identifiers (UFIDs). Operations: Read, Write, Create, Delete, GetAttributes, SetAttributes.
- **Directory Service:** Maps text names to UFIDs. Provides a hierarchical namespace. Translates human-readable paths to internal file IDs.
- **Client Module:** Runs on each client machine. Integrates flat file service and directory service. Provides familiar file API (open, read, write, close) to user programs.

3.3 Sun NFS (Network File System)

NFS is the most widely deployed distributed file system, originally developed by Sun Microsystems.

- **Stateless Server Design:** NFS servers do not maintain session state. Each request is self-contained. Simplifies crash recovery — client just retries.
- **NFS Protocol:** Based on RPC. Operations: LOOKUP, READ, WRITE, CREATE, REMOVE, RENAME, MKDIR.
- **Mount Protocol:** Client mounts a remote directory into its local file hierarchy.
- **VFS (Virtual File System):** OS abstraction layer that lets NFS appear as a local file system.
- **Caching:** Client-side caching of file blocks reduces network traffic. Cache consistency managed using timestamps.

Exam Tip: NFS stateless design is a favourite exam topic. Know WHY stateless is better for fault tolerance.

3.4 Naming Services

A **naming service** maps names (human-readable) to attributes (addresses, file handles, services). It is the cornerstone of any distributed system.

- **Name Space:** The set of all valid names. Can be flat (no structure) or hierarchical (tree-based).
- **Name Resolution:** Process of mapping a name to its associated value/attribute.
- **Binding:** Association between a name and its attributes.

3.5 DNS — Domain Name System

DNS is the largest, most widely used distributed naming system. It translates domain names (www.example.com) to IP addresses.

Root Servers	Top of hierarchy. Know location of all TLD servers. 13 logical root servers globally.
TLD Servers	Manage top-level domains (.com, .org, .in). Know location of authoritative servers.
Authoritative	Hold actual DNS records for a domain (A, CNAME, MX, NS records).

Local/Resolver

ISP-provided resolver that performs DNS lookups on behalf of clients.

- **Iterative Resolution:** Resolver asks each server in turn. Server returns referral, resolver queries next. Client does the work.
- **Recursive Resolution:** Resolver asks local server; local server does all querying and returns final answer to client.
- **Caching:** Responses cached per TTL (Time To Live) to reduce load. Enables fast repeated lookups.
- **Zone Transfer:** Primary DNS server pushes data to secondary DNS servers for redundancy.

Key Point: Iterative: client talks to each server. Recursive: server does all the work for the client. Iterative reduces load on root servers.

4.1 Physical Clocks & Synchronization

- **Clock Drift:** Each machine's clock runs at slightly different rate. Clocks diverge over time.
- **Clock Skew:** Difference in clock values between two machines at a given instant.
- **Cristian's Algorithm:** Client asks time server for current time T . Accounts for round-trip delay. Estimated time = $T + (RTT/2)$.
- **Berkeley Algorithm:** Time daemon asks all machines for their times, computes average, sends corrections.
- **NTP (Network Time Protocol):** Hierarchical. Stratum-0 = atomic clocks. Stratum-1 syncs to stratum-0. Widely used on internet.

4.2 Logical Clocks — Lamport Timestamps

Since physical clock synchronization is imperfect, Lamport introduced **logical clocks** to capture causal ordering of events.

Happens-Before Relation ($\rightarrow$):

- If a and b are events in the same process and a comes before b , then $a \rightarrow b$.
- If a is the sending of a message and b is its receipt, then $a \rightarrow b$.
- Transitivity: if $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.
- If neither $a \rightarrow b$ nor $b \rightarrow a$, events are **concurrent**.

Lamport Clock Rules:

- Each process maintains counter LC , initially 0.
- Before each event, $LC = LC + 1$.
- When sending message, include current LC in message.
- On receiving message with timestamp T : $LC = \max(LC, T) + 1$.

Key Point: Lamport clocks: if $a \rightarrow b$ then $LC(a) < LC(b)$. But $LC(a) < LC(b)$ does NOT imply $a \rightarrow b$. They cannot detect concurrency.

4.3 Vector Clocks

Vector clocks solve Lamport's limitation by capturing **causality precisely**.

- Each process P_i maintains vector $VC[1..n]$ where n = number of processes.
- $VC[i]$ = number of events P_i has experienced.
- On event: $VC[i] = VC[i] + 1$.
- On send: include entire VC vector in message.
- On receive from P_j with vector W : $VC[k] = \max(VC[k], W[k])$ for all k ; then $VC[i] = VC[i] + 1$.
- $VC(a) < VC(b)$ ■ a causally precedes b . Equal in some positions and less in others $\rightarrow$ concurrent.

Exam Tip: Draw a space-time diagram and trace vector clock values. This is a frequent 10-mark problem.

4.4 Global State & Chandy-Lamport Snapshot Algorithm

A **consistent global state** (consistent cut) records local states of all processes and messages in transit such that no message is shown received without being shown sent.

- **Chandy-Lamport Algorithm:** Process P_i records its local state, sends **MARKER** on all outgoing channels, records incoming messages on each channel until **MARKER** received on that channel.
- Used for: detecting deadlocks, debugging, checkpointing for fault tolerance.

4.5 Distributed Mutual Exclusion

Centralized Algorithm	One coordinator grants permission. Simple but single point of failure. Bottleneck at coordinator.
Token Ring	Processes arranged in logical ring. Token circulates; holder can enter CS. No starvation. Wasted bandwidth if no one wants CS.

Ricart-Agrawala	Fully distributed. Process broadcasts REQUEST with Lamport timestamp. Enters CS after receiving OK from all. On receiving REQUEST: send OK if not interested; else defer if your timestamp is lower.
------------------------	--

4.6 Election Algorithms

- **Bully Algorithm:** Process with highest ID wins. When a process notices coordinator has failed, it sends ELECTION to all higher-ID processes. If no response, it becomes coordinator. Announces via COORDINATOR message.
- **Ring Algorithm:** Processes in logical ring. Initiator sends ELECTION with its ID. Each process forwards with max(received ID, own ID). Initiator receives its own ID back → it is the winner.

4.7 Multicast Communication

FIFO Ordering	Messages from same sender received in send order.
Causal Ordering	If send(m1) → send(m2) then every process receives m1 before m2.
Total Ordering	All processes receive all messages in same order (strongest guarantee).

4.8 Authentication & Security

- **Symmetric Cryptography (AES, DES):** Same key for encrypt and decrypt. Fast. Key distribution problem.
- **Asymmetric Cryptography (RSA):** Public key to encrypt, private key to decrypt. Solves key distribution. Slower.
- **Digital Signatures:** Sender signs with private key; receiver verifies with sender's public key.
- **Kerberos:** Ticket-based authentication. Trusted third party (KDC). Avoids sending passwords over network.
- **Password-based Auth:** Store hashed passwords (SHA-256 + salt). Never store plaintext.

Key Point: Consensus Problem (FLP Impossibility): In an asynchronous system, it is **IMPOSSIBLE** to achieve consensus if even one process may fail. This is a fundamental theoretical result.

5.1 Distributed Shared Memory (DSM)

DSM provides an abstraction of a single shared address space across multiple machines that do NOT physically share memory. Programs written for shared memory can run on distributed systems.

- **Implementation:** Memory divided into pages. When a process accesses a non-local page, a page fault triggers fetching the page from the owning node.
- **Advantage:** Programmer uses familiar shared-memory programming model.
- **Disadvantage:** False sharing, high latency for remote access, coherence overhead.

5.2 Consistency Models

Sequential Consistency	All processes see ALL memory operations in the same order. The order must be consistent with program order of each process. Strongest practical model.
Release Consistency	Synchronization only at acquire/release operations (lock/unlock). Changes propagated only at release. Reduces network traffic.
Entry Consistency	Each shared variable associated with a synchronization variable. Only relevant variables synchronized at acquire. Most relaxed.
Causal Consistency	Causally related writes seen by all in same order. Concurrent writes may be seen differently.

5.3 Case Studies: Ivy & Munin

- **Ivy (Sequential Consistency):** Page-based DSM. Single owner per page. On write, invalidate all copies. Fetch fresh copy on fault. Simple but high invalidation traffic.
- **Munin (Lazy Release Consistency):** Uses diff-based updates. Propagates changes only at release. Multiple writers allowed. Uses write notices and diffs to update remote copies efficiently.

5.4 CORBA — Common Object Request Broker Architecture

CORBA is a middleware standard that enables communication between objects written in different languages running on different machines.

IDL	Interface Definition Language. Language-neutral interface specification. Compiled to stubs/skeletons.
ORB	Object Request Broker. Core middleware that routes method calls to the correct object.
Stub	Client-side proxy generated from IDL. Marshals calls into IIOP messages.
Skeleton	Server-side proxy. Unmarshals IIOP messages and calls actual object method.
IIOP	Internet Inter-ORB Protocol. Standard wire protocol for CORBA over TCP/IP.
Naming Service	Maps object names to object references. Like a directory for CORBA objects.

Exam Tip: CORBA architecture diagram: Client → IDL Stub → ORB → IIOP → ORB → Skeleton → Server Object. Draw and label this for 10-mark questions.

5.5 Cloud-Native Distributed Systems

- **Containers (Docker):** Lightweight, portable packages containing application + dependencies. Isolated using Linux namespaces and cgroups. Faster than VMs.
- **Kubernetes (K8s):** Container orchestration platform. Automates deployment, scaling, and management. Key concepts: Pod (smallest unit), Service (networking), Deployment (manages replicas), Node (machine running pods).
- **Container vs VM:** Containers share host OS kernel (lighter). VMs have full OS (heavier, stronger isolation).

5.6 Microservices Architecture

Microservices decompose a monolithic application into small, independently deployable services that communicate over a network.

Service Independence	Each service has its own database, codebase, and deployment pipeline.
Communication	REST (HTTP/JSON) for synchronous; message queues for asynchronous communication.
API Gateway	Single entry point for clients. Routes requests to appropriate microservices.
Service Discovery	Services register themselves; clients discover them dynamically (e.g., Consul, Eureka).
Fault Isolation	Failure in one service does not cascade to others (circuit breaker pattern).

5.7 Middleware & Message Queues

Message-Oriented Middleware (MOM) decouples producers and consumers through a message broker.

Apache Kafka	Log-based, distributed streaming platform. Topics divided into partitions. Messages retained for configurable time. High throughput. Best for: event streaming, log aggregation, real-time analytics.
RabbitMQ	AMQP-based message broker. Supports complex routing (direct, fanout, topic, header exchanges). Messages deleted after consumption. Best for: task queues, RPC, request-reply patterns.

- **Kafka vs RabbitMQ:** Kafka retains messages (consumers can replay). RabbitMQ deletes after consumption. Kafka is pull-based; RabbitMQ is push-based.
- **Producer-Consumer Model:** Producer publishes to topic/queue. Consumer subscribes and processes messages asynchronously.

Key Point: Kafka = high-throughput streaming + replay capability. RabbitMQ = flexible routing + acknowledgment-based delivery.

Quick Revision — Module Summary

Module	Key Topics	Must-Know
Module 1	DS definition, Transparency, IPC, Marshalling, Sockets	6 Transparency types, TCP vs UDP
Module 2	RPC, Java RMI, Events, OS support, Quorum	RPC stub/skeleton mechanism, R+W>N
Module 3	DFS, NFS (stateless), DNS iterative/recursive	DNS resolution steps, NFS stateless design
Module 4	Lamport clocks, Vector clocks, Mutual exclusion, Elections, Atomic clock causality, Bully/Ring election, Ricart-Agrawa	
Module 5	DSM, Consistency models, CORBA, Microservices, Kafka/RabbitMQ	6.1.1 IDL/ORB/IOP, Kafka vs RabbitMQ